

Clean Code

VSCode Java & C++ 개발자를 위한 실전 클린코드 가이드



클린코드
개념 & 원칙



네이밍 &
스타일



함수 & 클래스
설계



리팩토링 &
코드스멜



단위 테스트 &
정적 분석



Java — JUnit 5 · Checkstyle · SpotBugs ·
PMD



C++ — GoogleTest · clang-tidy · cppcheck

전체 강의 구성 (3시간)

파트	시간	주제	핵심 내용
1부	30분	클린코드란? — 개념과 필요성	정의, Boy Scout Rule, 비용 곡선, 주요 원칙
2부	30분	네이밍 & 코딩 스타일	의도 표현, 일관성, Java vs C++ 네이밍 규칙
3부	30분	함수 & 클래스 설계 원칙	SRP · OCP · 함수 크기 · 추상화 수준
—	5분	휴식	—
4부	25분	리팩토링 & 코드스멜	코드스멜 식별, Extract Method, 안전한 리팩토링
5부	30분	단위 테스트 & TDD	JUnit 5 · GoogleTest · TDD 사이클 실습
6부	20분	정적 분석 도구 실전	Checkstyle · SpotBugs · PMD / clang-tidy · cppcheck
7부	5분	마무리 — 실천 로드맵	단계별 적용 전략, Q&A

1

클린코드란?

정의 · 필요성 · Boy Scout Rule · 원칙

🕒 00:00 ~ 00:30 (30분)

다음 중 클린 코드와 가장 거리가 먼 것은?

문제

1. 적절한 행 길이를 유지하고 있는 코드
2. 한 클래스에 모든 레이어의 기능을 작성한 코드
3. 잘 작동하고 있는 테스트 코드를 가지고 있는 코드
4. 하나의 동작을 담당하도록 설계된 코드

답

1. 적절한 행 길이를 유지하고 있는 코드
 2. 한 클래스에 모든 레이어의 기능을 작성한 코드
 3. 잘 작동하고 있는 테스트 코드를 가지고 있는 코드
 4. 하나의 동작을 담당하도록 설계된 코드
- 

실습: 어느 코드가 클린코드인가? (Java 예제)

같은 기능, 다른 설계 — 1분간 코드를 검토하고 어느 쪽이 클린코드인지 생각해 보세요

✗ List1 — 나쁜 코드

```
public class List1 {
    private Object[] elements = new Object[10];
    private boolean readOnly;
    private int size = 0;
    public void add(Object element) {
        if (!readOnly) {
            int newSize = size + 1;
            if (newSize > elements.length) {
                // 배열 크기 확장 로직...
            }
            elements[size] = element; size++;
        }
    }
}
```

List2가 클린코드인 이유:

- ① 가독성 — 메서드 분리로 의도 명확
- ② SRP — 각 메서드가 하나의 책임
- ③ 매직 넘버 제거 — 상수 사용으로 의미 명확
- ④ 테스트 용이성 — 메서드별 독립 테스트 가능

✓ List2 — 클린코드

```
public class List2 {
    private static final int INCREMENT = 10;
    private static final int INIT_SIZE = 10;
    private Object[] elements = new
Object[INIT_SIZE];
    private boolean readOnly;
    private int size = 0;
    public void add(Object element) {
        if (readOnly) return;
        if (isElementStoreFull())
            increaseElementStore();
        addElementAtEnd(element);
    }
    private boolean isElementStoreFull() {...}
    private void increaseElementStore() {...}
    private void addElementAtEnd(...) {...}
}
```

클린코드란 무엇인가?

Bjarne Stroustrup
(C++ 창시자)

" 나는 우아하고 효율적인 코드를 좋아한다. 논리가 간단해야 버그가 숨어들지 못한다.
깨끗한 코드는 한 가지를 제대로 한다. "

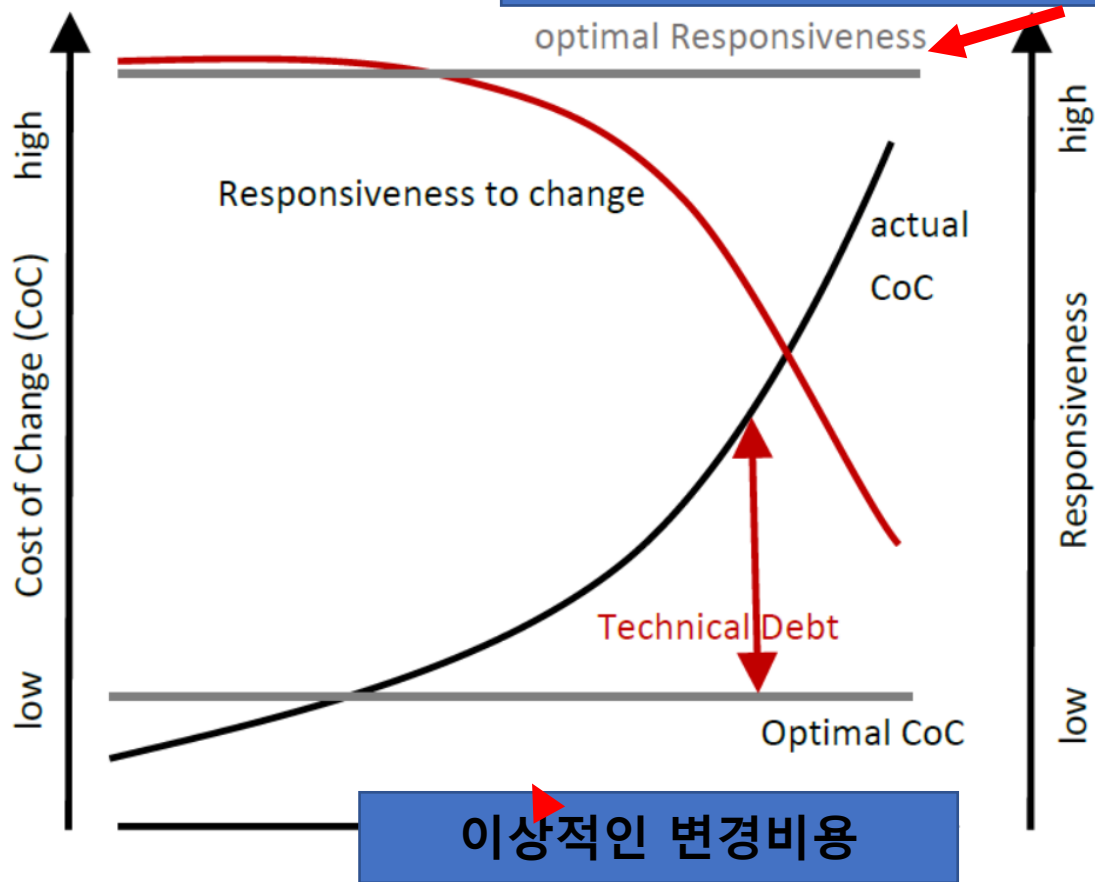
Grady Booch
(OOP 분석설계 저자)

" 깨끗한 코드는 단순하고 직접적이다. 깨끗한 코드는 잘 쓴 문장처럼 읽힌다.
명쾌한 추상화와 단순한 제어문으로 가득하다. "

Dave Thomas
(Agile 선언문 저자)

" 단위 테스트 케이스와 인수 테스트 케이스가 존재한다.
의존성은 최소이며 각 의존성을 명확히 정의한다. "

클린코드를 작성해야 하는 이유



클린코드를 작성해야 하는 이유

시간이 지날수록...

변경 비용

클린코드 ❌ → 기하급수적 증가

클린코드 ✅ → 완만하게 유지

응답 속도

클린코드 ❌ → 점점 느려짐

클린코드 ✅ → 빠른 응대 유지

버그 발생

클린코드 ❌ → 수정 시 부작용

클린코드 ✅ → 테스트로 즉시 감지

우리가 자주 접하는 나쁜 코드 패턴

- 부적절한 네이밍 (a, tmp, data...)
- 하나의 함수에 여러 로직이 뒤섞임
- 비대한 클래스 · 비대한 메서드
- 의미를 알기 어려운 조건문 · 매직 넘버
- 사용하지 않는 변수 · 중복 코드
- 테스트 없이 수정 → 부작용 공포

💡 대부분의 결함은 기존 코드 수정 시 발생합니다. 이해하기 쉬운 코드는 오류 위험을 최소화합니다.

Boy Scout Rule & 클린코드 핵심 원칙



Boy Scout Rule — "캠프장은 처음 왔을 때보다 더 깨끗하게 해놓고 떠나라"

Follow Standard Conventions

코딩 표준·아키텍처·설계 가이드 준수

Java: Google Style / Sun Style

C++: .clang-tidy / LLVM Style

KISS — Keep It Simple, Stupid

단순한 것이 효율적

복잡한 로직은 함수로 분리

한 함수 = 한 가지 역할

Root Cause Analysis

근본 원인을 찾아라

표면만 수정하면 반복됨

정적 분석으로 패턴 파악

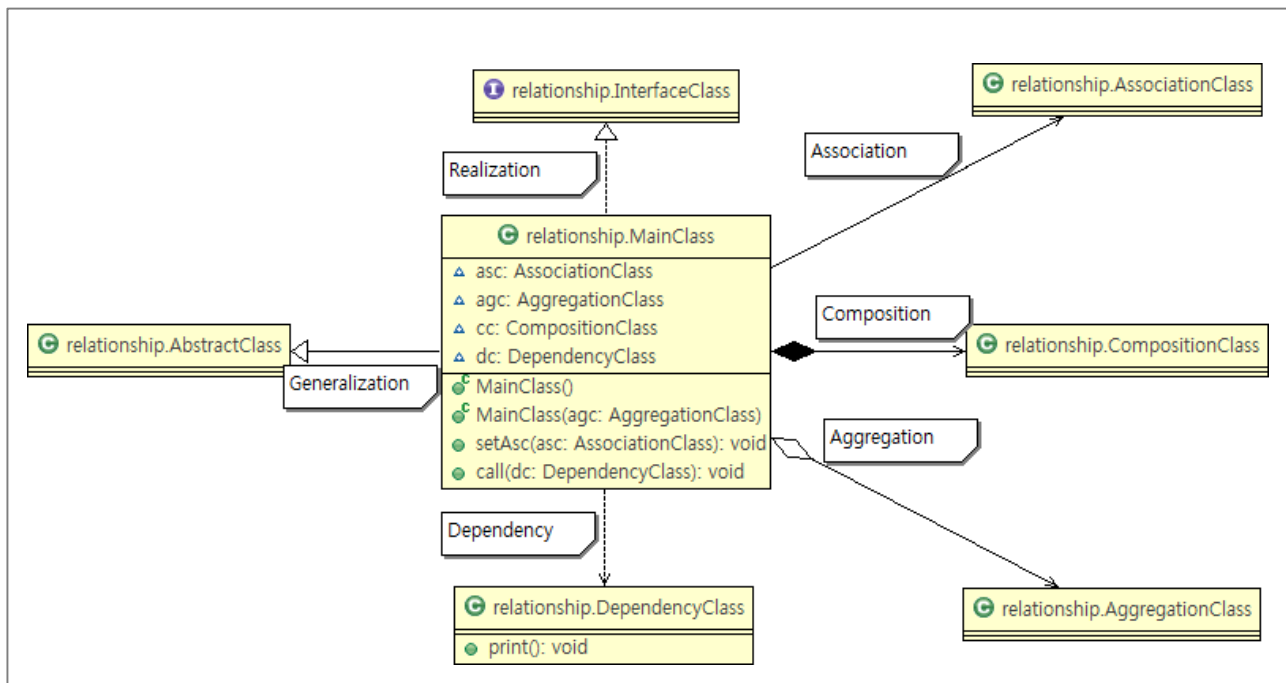
DRY — Don't Repeat Yourself

중복 코드는 유지보수 지옥

Extract Method / 공통 유틸로 분리

PMD CPD로 중복 탐지

객체의 관계 표현 방법은 결국 객체간의 의존성에 대한 설계이다.



객체 지향 프로그래밍의 주요원칙 (Class Design)

S

Single
Responsibility
Principle (SRP)

하나의 클래스는
하나의 책임만
가져야 한다.

O

Open/Closed
Principle
(OCP)

클래스는 확장에
대하여 열려
있어야 하고,
변경에 대해서는
닫혀 있어야 한다.

L

Liskov
Substitution
Principle (LSP)

파생 클래스의
메소드는 기반
클래스의
메서드를
대체하여 사용될
수 있어야 한다.

I

Interface
Segregation
Principle (ISP)

클라이언트가
사용하지 않는
메소드에
의존하지 않아야
한다.

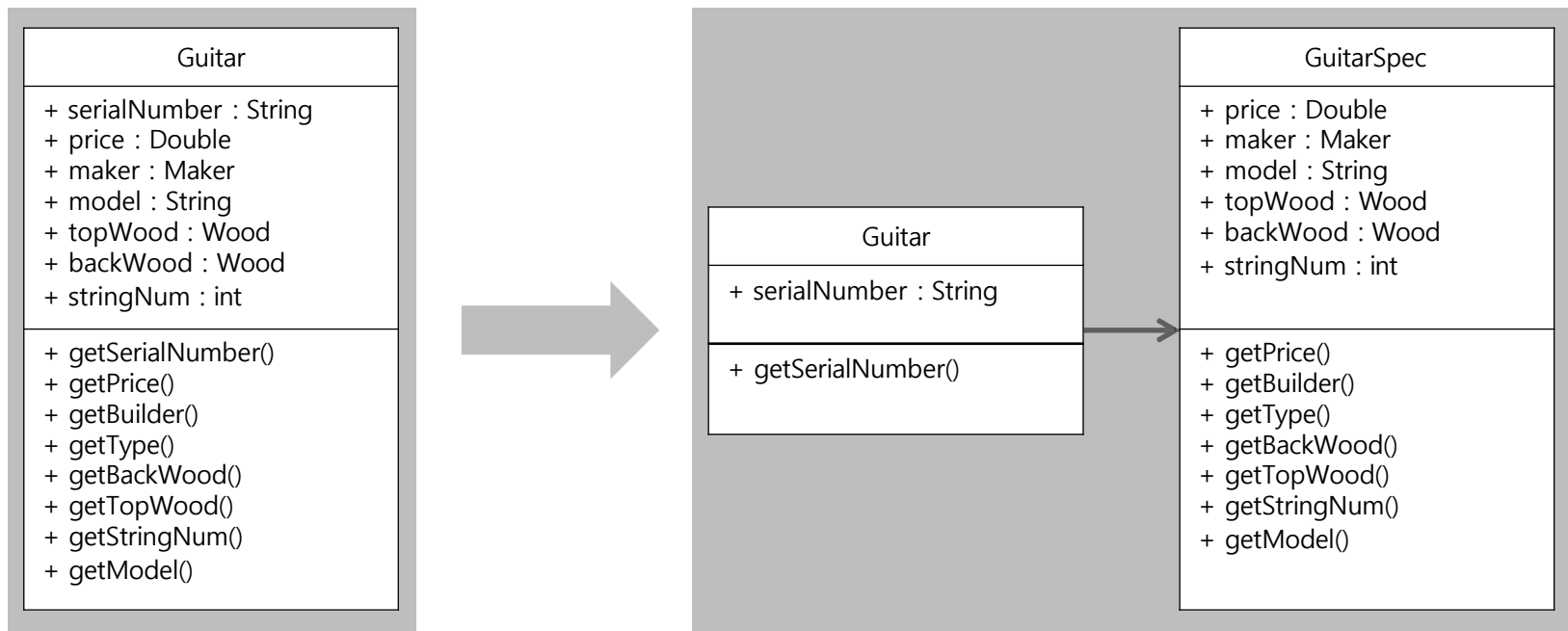
D

Dependency
Inversion
Principle (DIP)

추상화된 것은
구체적인 것에
의존하면 안된다.
(자주 변경되는
구체적인 것에
의존하지 말고
추상화된 것을
참조)

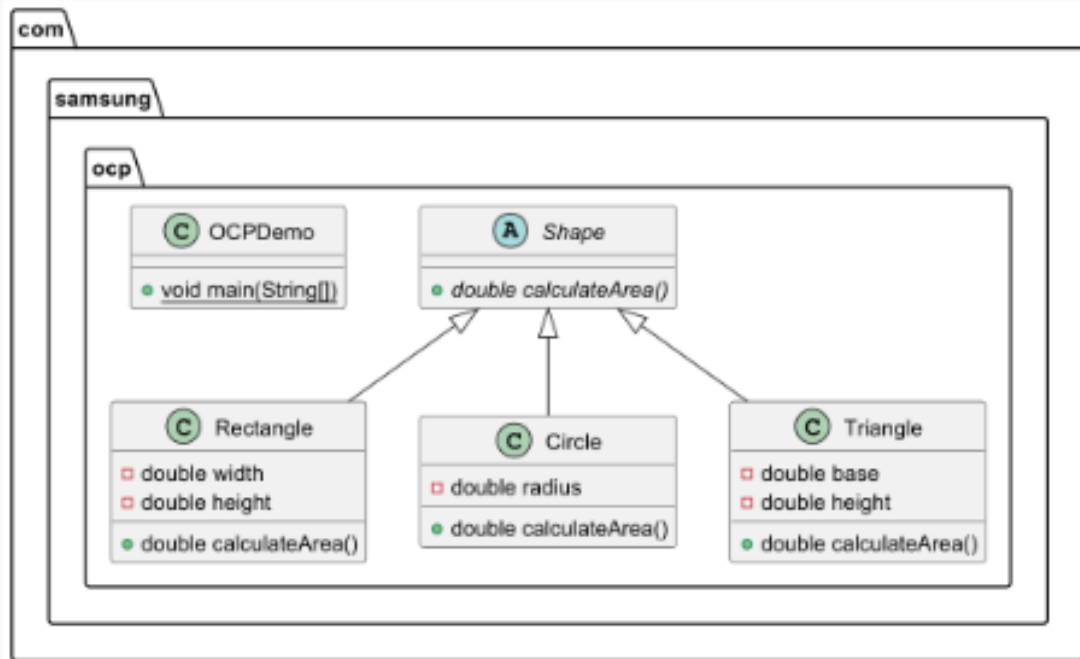
객체 지향 프로그래밍의 주요원칙 (Class Design)

SRP, Single Responsibility Principle



객체 지향 프로그래밍의 주요원칙 (Class Design)

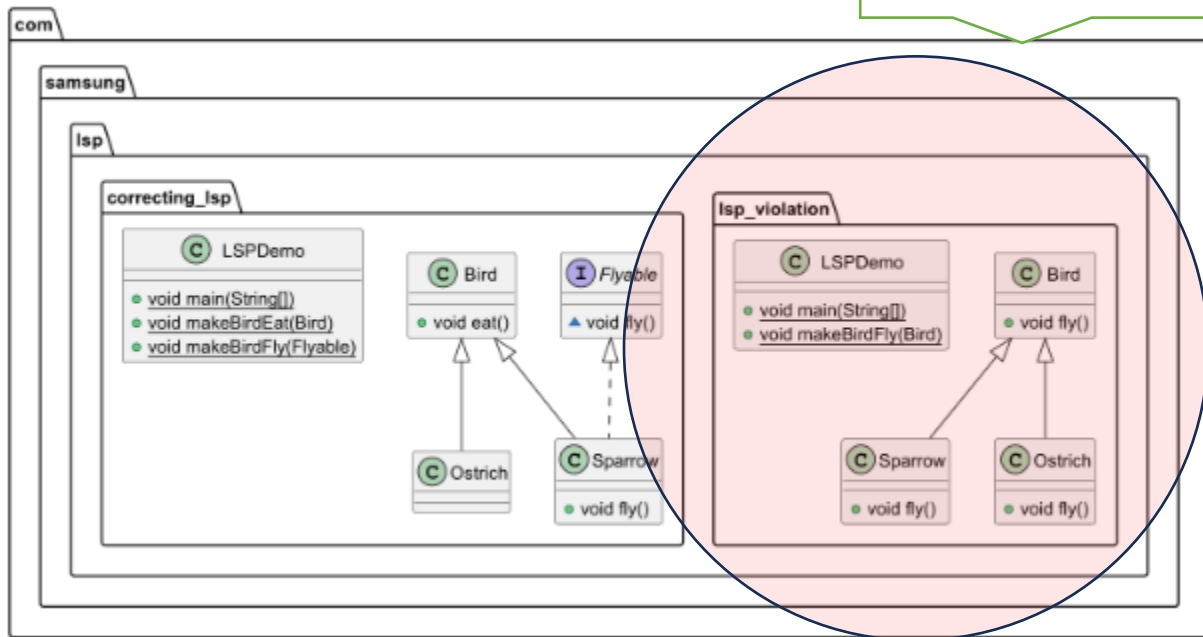
OCP, Open/Closed Principle



객체 지향 프로그래밍의 주요원칙 (Class Design)

Liskov Substitution Principle, LSP

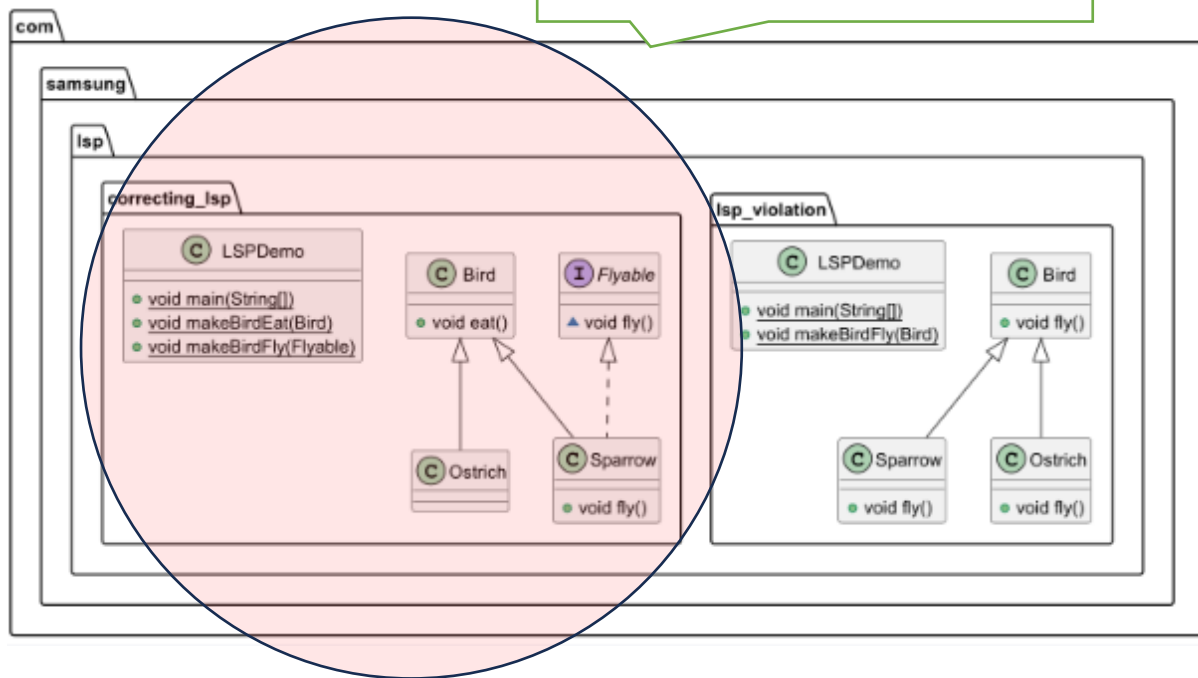
잘못된 예: LSP 위반



객체 지향 프로그래밍의 주요원칙 (Class Design)

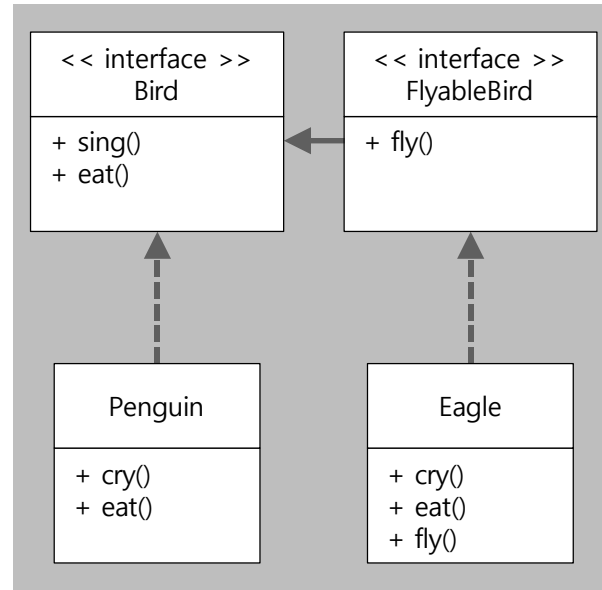
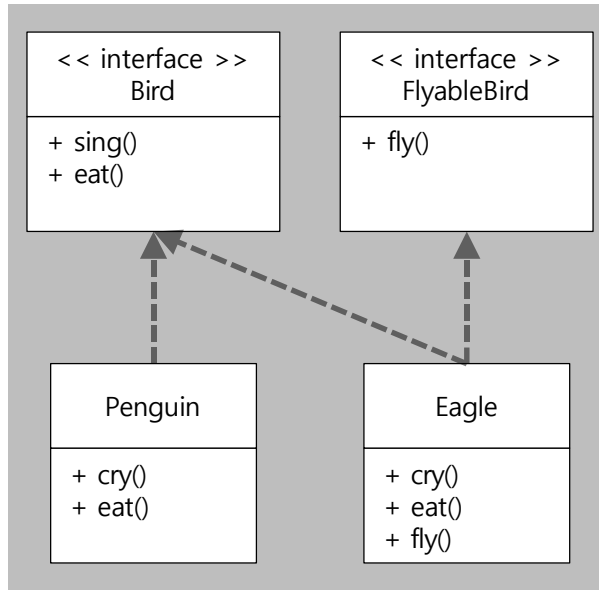
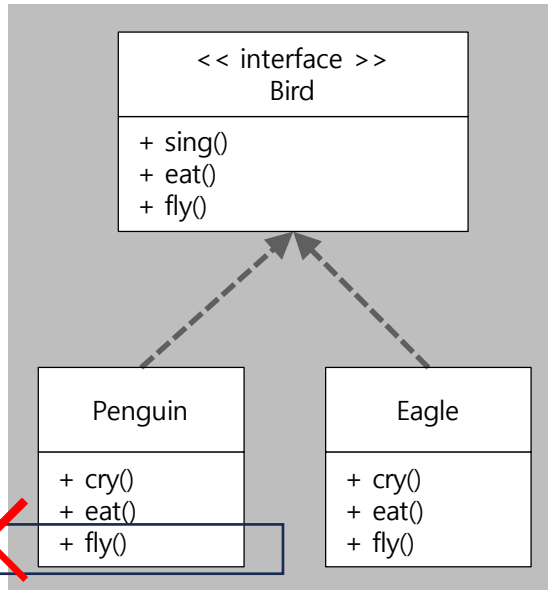
Liskov Substitution Principle, LSP

수정된 코드: LSP 준수



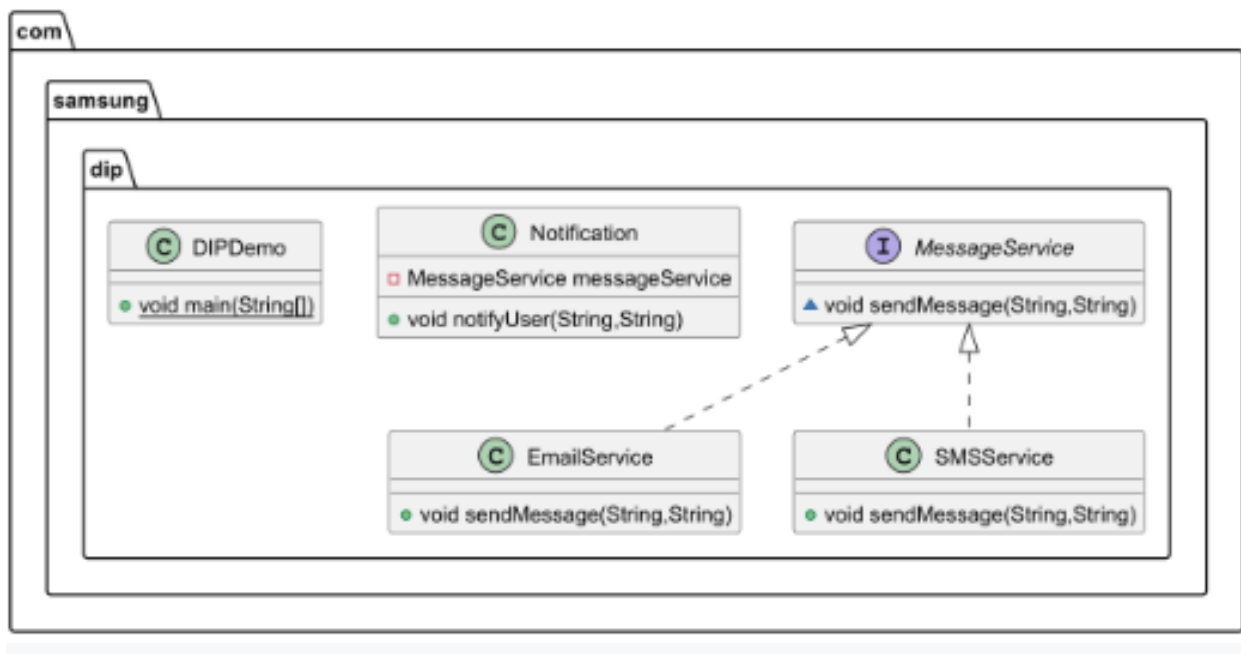
객체 지향 프로그래밍의 주요원칙 (Class Design)

Interface Segregation Principle, ISP



객체 지향 프로그래밍의 주요원칙 (Class Design)

Dependency Inversion Principle, DIP



2

네이밍 & 코딩 스타일

의도를 표현하는 이름 · 일관성 · Java vs C++ 규칙

🕒 00:30 ~ 01:00 (30분)

좋은 네이밍의 원칙

의도를 표현하라

✗ `int d; // 경과 시간(일)`

✓ `int elapsedTimeInDays;`

그릇된 정보 피하기

✗ `accountList` – List 아닐 수도

✓ `accountGroup` /
`bunchOfAccounts`

의미있는 구분

✗ `a1, a2` / `ProductInfo`,
`ProductData`

✓ `source, destination` / `Product`

검색 가능한 이름

✗ `if (s == 5) { ... }`

✓ `final int WORK_DAYS = 5;`

클래스 = 명사

✗ `class Manager` / `class`
`Processor`

✓ `class Customer` / `class`
`Account`

메서드 = 동사

✗ `name()` / `age()`

✓ `getName()` / `setAge()` /
`isValid()`

Java vs C++ 네이밍 & 스타일 비교

구분	 Java (Google Style)	 C++ (LLVM / Google Style)
클래스	PascalCase: CalculatorService	PascalCase: CalculatorService
메서드 / 함수	camelCase: calculateTotal()	camelCase or snake_case: calculateTotal()
변수	camelCase: elapsedTime	snake_case: elapsed_time
상수	UPPER_SNAKE: MAX_SIZE	kCamelCase or UPPER_SNAKE: kMaxSize
멤버 변수	camelCase: userName	trailing underscore: user_name_
패키지 / 네임스페이스	lowercase: com.example.util	lowercase: com::example::util
들여쓰기	2 또는 4 스페이스	2 또는 4 스페이스 (clang-format)
최대 라인 길이	100 문자 (Google Java Style)	80~120 문자 (clang-format 설정)

 VSCode: Java → Checkstyle, C++ → clang-format (.clang-format 파일) 으로 스타일을 자동 강제할 수 있습니다.

Java vs C++ 네이밍 & 스타일 비교

몇 가지 코딩 스타일

- Google Style Guides
 - [https://google.github.io/styleguide/
styleguide](https://google.github.io/styleguide/styleguide)
-

Google Style Guides

Every major open-source project has its own style guide: a set of conventions (sometimes arbitrary) about how to write code for that project. It is much easier to understand a large codebase when all the code in it is in a consistent style.

“Style” covers a lot of ground, from “use camelCase for variable names” to “never use global variables” to “never use exceptions.” This project ([google/styleguide](https://google.github.io/styleguide)) links to the style guidelines we use for Google code. If you are modifying a project that originated at Google, you may be pointed to this page to see the style guides that apply to that project.

- [AngularJS Style Guide](#)
- [Common Lisp Style Guide](#)
- [C++ Style Guide](#)

Java vs C++ 네이밍 & 스타일 비교

변수·함수·유형 표기 방식

- 카멜 표기법(Camel case)

- camelCase

- 좀 더 공식화된 용어로는 중간 대문자(medial caps)라는 용어도 있다. '카멜'은 '낙타'를 뜻한다. '카멜백 표기법(camelback case)'이라고도 불렀지만 요즘은 거의 쓰이지 않는다. 낙타의 등에 혹이 불룩하게 나와 있는 것에서 따온 이름으로, 각 단어의 첫 글자를 대문자로 표기하되 첫 단어의 첫 글자는 소문자로 표기하는 방식이다. Java가 변수나 함수 이름에 이를 사용하고 있으며, 자바스크립트, C#, Dart와 같은 많은 언어들이 사용하고 있다.

- 파스칼 표기법(Pascal case)

- PascalCase

- 각 단어의 첫 글자를 대문자로 표기한다. 첫 단어의 첫 글자도 대문자로 표기한다. 요즈음은 그냥 이것도 카멜 표기법이라고 부른다. 구분해서 부를 때는 이쪽은 대문자 카멜 표기법(upper camel case)으로, 첫 단어 첫 글자를 소문자로 쓰는 방식은 소문자 카멜 표기법(lower camel case)이라고 부른다. 파스칼 언어에서 이러한 표기법을 사용했기 때문에 붙은 이름이다. 변수나 함수 이름으로는 카멜백 표기법을 사용하는 언어라고 해도 적어도 클래스 이름은 파스칼 표기법을 사용하는 게 보통이다. C#은 변수 이름만 카멜 표기법을 쓰며 메서드 이름이나 클래스 이름은 파스칼 표기법을 쓴다. 또는 기본 유형은 카멜 표기법 또는 스네이크 표기법을 사용하더라도 사용자 정의 유형은 파스칼 표기법을 사용하기도 한다. 당연히 파스칼은 변수 이름도 파스칼 표기법을 사용한다.

Java vs C++ 네이밍 & 스타일 비교

변수·함수·유형 표기 방식

- 스네이크 표기법(Snake case)

- SNAKE_CASE, snake_case

- 땅바닥을 기어다니는 뱀의 모양과 비슷하다고 해서 붙은 이름. 모두 소문자(혹은 모두 대문자)로 쓰되, 띄어쓰기를 밑줄(_) 기호로 대체하는 것이다. C, C++과 같은 언어들이 이 방법을 사용한다. 변수나 함수, 유형, 클래스 이름은 소문자를, 상수 및 매크로 이름은 대문자를 사용하는 게 관례. Java도 static final 변수(사실상 상수)와 enum 값 이름에 대문자 스네이크 표기법을 쓴다. Python은 변수나 함수 이름은 스네이크 표기법을, 클래스 이름은 파스칼 표기법을 쓰는 관례를 사용한다. 띄어쓰기를 하면 안 되는 파일 이름을 적을 때에도 이 방법을 사용한다. 포트란이나 코볼처럼 문자열을 제외하고 대소문자 구별을 안 하는 언어라면 스네이크 표기법.

- 케밥 표기법(Kebab case)

- KEBAB-CASE, kebab-case

- 케밥 꼬치의 모양을 보고 따온 말로, 꼬치에 재료가 듬성듬성 꽂혀 있는 것과 비슷한 모양이라는 의미다. 모두 소문자(혹은 모두 대문자)로 쓰되, 띄어쓰기를 하이픈(-) 기호로 대체하는 것이다. 하이픈은 뉘셈 기호로도 쓰이기 때문에 일반 프로그래밍 언어에서는 이름에 잘 안 쓰는 편이며 많은 언어가 변수나 함수, 유형 이름에 하이픈을 못 쓰게 한다. CSS는 연산을 하지 않으므로 이 방식을 사용한다. HTML도 id나 class의 이름은 케밥 표기법을 쓴다. 이들은 주로 CSS에서 쓰이기 때문. URL에도 종종 쓰이며, 띄어쓰기를 하면 안 되는 파일 이름에도 가끔 쓰인다.

Java vs C++ 네이밍 & 스타일 비교

변수·함수·유형 표기 방식

- 헝가리안 표기법(Hungarian notation)

- strHungarianNotation

- 영어로는 대소문자를 뜻하는 case 대신에 표기법을 뜻하는 notation을 사용하는데, 이는 대소문자 문제가 아니기 때문이다. 파스칼 표기법처럼 첫 단어를 포함해서 모든 단어의 첫 글자를 대문자로 하되, 그 앞에 변수의 유형을 뜻하는 약어를 소문자로 붙여 준다. 정수(int)형이라면 i, 문자열(string)이라면 str을 붙이는 식이다. 이 방법을 처음 고안한 마이크로소프트의 개발자[이와 같은 표기법을 생각한 것은 MS로 오기 전 제록스의 팔로-알토 연구소에 있었을 때라고 한다]이와 같은 표기법을 생각한 것은 MS로 오기 전 제록스의 팔로-알토 연구소에 있었을 때]가 헝가리인이었기 때문에 이런 이름이 붙었다. 윈도우 API에서 이 방식을 사용했으며, MFC에도 썼지만 지금은 잘 안 쓰는 방식이다. 마이크로소프트조차도 이전 윈도우 API와 호환성을 위해서 남겨놓았을 뿐, 더 이상 쓰지 않으며 .NET 프레임워크 프로그래밍에 사용하는 일반 명명 규칙에서 아예 쓰지 말라고 못박았다. 그래도 요즘도 가끔 쓰이는 경우는 있다. 구글은 한때 글로벌 상수 이름 앞에 소문자 k를 붙였는데, 최근에는 가이드라인을 통해 이렇게 하지 말라고 권고하고 있지만 오픈 소스 저장소에 올라와 있는 구글 코드 중에는 여전히 이를 사용한 코드가 종종 보인다. 단, 구글의 C++ 코딩 가이드라인에서는 전역 상수 앞에 k를 붙이도록 하고 있다.

실습 - Java vs C++ 네이밍 & 스타일 비교

과제 1: Java vs C++ 네이밍 & 스타일 비교

제시
문제

VSCode: Java → Checkstyle

제시
문제

VSCode: C++ → clang-format (.clang-format 파일)

주석 — 좋은 주석 vs 나쁜 주석

코드만으로 표현할 수 없을 때만 주석을 작성하라. 나쁜 주석은 코드를 읽는 시간을 두 배로 만든다.

✓ 좋은 주석

```

// 코드의 '왜'를 설명 - 의도 주석
// 소수를 구하기 위해 에라토스테네스 체 사용
// (단순 반복보다 O(n log log n)으로 빠름)

// TODO: 성능 최적화 필요 (이슈 #1234)

/**
 * @param days 기준일로부터 경과 일수 (양수)
 * @throws IllegalArgumentException days < 0
 */
public Date addDays(int days) { ... }
```

✗ 나쁜 주석

```

// 나쁜 주석 유형

// 변수 선언 (무의미한 주석)
int count = 0;

// i를 1 증가시킨다 (코드 번역)
i++;

// 오래된 코드 주석으로 남기기 (죽은 코드)
// for (int j = 0; j < 10; j++) ...
```

// 이렇게 하면 됩니다 (불필요한 주석)



주석이 필요하다면 코드가 자기 설명적이지 않다는 신호입니다. 먼저 코드를 개선하고, 주석은 최후의 수단으로 활용하세요.

3

함수 & 클래스 설계 원칙

SRP · 함수 크기 · 추상화 수준 · 오류 처리

🕒 01:00 ~ 01:30 (30분)

함수 설계 — 작게, 하나만, 한 수준으로

1 작게 만들어라

함수는 20줄 이내가 이상적
블록과 들여쓰기 1~2단계 이내
중첩 구조가 깊으면 분리 신호

2 한 가지만 해라

함수 이름으로 설명되는
한 가지 작업만 수행
섹션을 나눌 수 있다면 분리 필요

3 추상화 수준 통일

한 함수 내 추상화 수준은 하나
고수준(비즈니스)과 저수준(I/O)
을 섞지 말 것

4 서술적 이름을 사용

길고 서술적인 이름이
짧고 의미없는 이름보다 낫다
testableHtml → setupTeardown...

5 인수를 최소화

이상적: 0개, 최대 3개
3개 초과 시 매개변수 객체 고려
boolean 인수는 지양

6 부수 효과 없애기

함수는 한 가지 일만
예측 불가능한 숨겨진 변경 금지
CQS: 명령과 조회를 분리

SRP — 단일 책임 원칙 (Single Responsibility Principle)

"클래스나 함수는 단 하나의 책임만 가져야 한다 — 변경의 이유가 하나여야 한다"

❌ SRP 위반 (Java)

```
// 한 클래스가 너무 많은 책임
public class Resume {
    public String introduceMe() {...}
    public String getGrade()    {...}
    public String toJson()      {...} // 직렬화?
    public String toXML()       {...} // 직렬화?
    public String toHtml()      {...} // UI?
}
```

⚙️ C++도 동일 원칙 적용

// SRP 위반 ❌

```
class Guitar {
    string serialNumber; // 식별 책임
    double price;        // 사양 책임
    Maker maker;         // 사양 책임
    string model;        // 사양 책임
};
```

// SRP 준수 ✅

```
class Guitar {
    string serialNumber;
    GuitarSpec spec; // 위임
};
class GuitarSpec {
    double price; Maker maker; string model;
```

✅ SRP 준수 (Java)

```
// 책임을 분리
public class Resume {
    public String introduceMe() {...}
    public String getGrade()    {...}
}
interface CustomConverter {
    String convert(Resume r);
}
class JsonConverter implements CustomConverter {...}
```

디미터 법칙(Law of Demeter: "최소한의 지식 원칙(The Principle of Least Knowledge))

3부

✓ 좋은 예 (준수 코드)

```
order.getCustomerCity();
```

그리고 내부 구현:

```
class Order {  
    private Customer customer;  
  
    public String getCustomerCity() {  
        return customer.getAddress().getCity();  
    }  
}
```

외부는 **Order만** 알면 됨

✗ 나쁜 예 (위반 코드)

```
order.getCustomer().getAddress().getCity();
```

문제:

order → customer → address → city

- 내부 구조를 다 알고 있음
- 구조 바뀌면 전부 깨짐

오류 처리 — Clean Error Handling

원칙	나쁜 코드 ❌	좋은 코드 ✅
오류 코드 대신 예외 사용	<code>if (deletePage(p) == E_OK) {...}</code>	<code>try { deletePage(p); } catch(...)</code>
null 반환하지 않기	<code>return null; // 호출자에 책임 떠넘김</code>	<code>return Optional.empty()</code> 또는 빈 컬렉션
null 전달하지 않기	<code>doSomething(null); // NPE 위험</code>	<code>assert p != null; / Optional 사용</code>
예외에 의미 담기	<code>throw new Exception("error");</code>	<code>throw new StorageException("파일 읽기 실패", e)</code>
미확인 예외 선호	<code>throws IOException, SQLException...</code>	RuntimeException 래핑으로 의존성 감소

Java Checked vs Unchecked Exception

```
// Checked: 컴파일러가 처리 강제 (IOException)
try { reader.readLine(); }
catch (IOException e) { log(e.getMessage()); }
// Unchecked: 런타임 예외 (NullPointerException)
```

C++ 예외 처리

```
// C++ 예외 - noexcept / std::optional 활용
std::optional<int> divide(int a, int b) {
    if (b == 0) return std::nullopt;
    return a / b; }
```

4

리팩토링 & 코드스멜

코드스멜 식별 · Extract Method · 안전한 리팩토링

🕒 01:35 ~ 02:00 (25분)

코드스멜 (Code Smell) 주요 유형

중복 코드

같은 코드가 두 곳 이상
→ Extract Method / 공통 유틸

긴 메서드

20줄 초과
→ 기능별 분리

거대한 클래스

너무 많은 책임
→ SRP 적용, 클래스 분리

긴 매개변수 목록

3개 초과
→ 매개변수 객체 도입

죽은 코드

사용하지 않는 코드
→ 즉시 삭제 (Git이 기억함)

매직 넘버 / 리터럴

의미 불명 숫자·문자열
→ 이름 있는 상수로 추출

깊은 중첩

if/for 3단계 이상
→ Early Return / 함수 추출

부적절한 이름

a, tmp, data, util...
→ 의도를 드러내는 이름

리팩토링 — Extract Method 실습 (Java & C++)

리팩토링: 외부 동작을 바꾸지 않으면서 내부 구조를 개선하는 것 | 항상 테스트 먼저!

✗ Before — Java

```
void printOwing(double amount) {
    // 배너 출력
    System.out.println("*****");
    System.out.println("* Customer Owing *");
    System.out.println("*****");
    // 세부사항 출력
    System.out.println("name: " + _name);
    System.out.println("amount: " + amount);
}
```

✓ After — Java

```
void printOwing(double amount) {
    printBanner();
    printDetails(amount); // 의도가 명확
}
void printBanner() { ... }
void printDetails(double amount) { ... }
```

⚙ C++ 동일 원칙

```
// Before: 함수에 모든 로직 집중
void processOrder(Order& o) {
    // 유효성 검사
    if (o.amount <= 0) throw
        std::invalid_argument("");
    if (o.customerId.empty()) throw ...;
    // 할인 계산
    double discount = o.amount > 1000 ? 0.1 : 0;
}
```

```
// After: 책임 분리
void processOrder(Order& o) {
    validateOrder(o);
    double discount = calculateDiscount(o);
    saveOrder(o, discount);
}
void validateOrder(const Order& o) { ... }
double calculateDiscount(const Order& o) { ... }
```

안전한 리팩토링 — 테스트 + 정적 분석의 역할

리팩토링 전 테스트 없이 코드를 고치는 것은 눈 감고 운전하는 것과 같습니다.

1 테스트 먼저 확인

기존 테스트 모두 통과 확인
JUnit 5 / GoogleTest 실행
테스트 없으면 먼저 작성

2 정적 분석 실행

리팩토링 전 경고 목록 저장
Java: mvn checkstyle:check
C++: clang-tidy --fix-errors

3 작은 단위로 변경

한 번에 하나씩 리팩토링
VSCode F5로 즉시 테스트
커밋 단위도 작게

4 테스트 재실행

변경 후 전체 테스트 재실행
실패 시 즉시 되돌리기
git stash / git revert

5 정적 분석 재실행

새 경고 발생 여부 확인
mvn spotbugs:check / pmd:check
cppcheck 재실행

6 커밋 & 코드 리뷰

작고 명확한 커밋 메시지
refactor: Extract Method for printBanner
PR에 변경 이유 설명

5

단위 테스트 & TDD

테스트의 가치 · JUnit 5 · GoogleTest · TDD 사이클

🕒 02:00 ~ 02:30 (30분)

단위 테스트가 중요한 이유

테스트 코드가 없는 코드를 수정한다면?

→ 개발자는 내가 고친 코드가 어떤 영향을 끼칠지 몰라 수정이 쉽지 않다. 결국 코드는 썩는다.

테스트 코드가 잘 갖춰진 코드를 수정한다면?

→ 코드를 잘못 고쳐도 테스트에서 잡아주니 편하게 수정할 수 있다. 리팩토링 자유도가 높아진다.

좋은 테스트의 F.I.R.S.T 원칙

원칙	설명	Java (JUnit 5)	C++ (GoogleTest)
Fast	테스트는 빨리 돌아야 한다	@Test — ms 단위	TEST() — ms 단위
Independent	테스트는 서로 의존하면 안 된다	@BeforeEach 초기화	SetUp() / TearDown()
Repeatable	어떤 환경에서도 같은 결과	@RepeatedTest	INstantiate_Test_Suite_P
Self-Validating	결과는 bool로 자동 판단	assertEquals(expected, actual)	EXPECT_EQ(expected, actual)
Timely	테스트는 실 코드 직전에 작성	TDD — @Test 먼저 작성	TDD — TEST() 먼저 작성

TDD 사이클 — Java & C++ 비교

 RED
실패하는 테스트 작성

 GREEN
최소 코드로 통과

 REFACTOR
코드 정리

 Java — JUnit 5

```
@Test @DisplayName("덧셈 검증")
void testAdd() { assertEquals(5, calc.add(2,3));
}
```

 C++ — GoogleTest

```
TEST(CalculatorTest, AddBasic) {
    EXPECT_EQ(5, calc.add(2, 3)); }
```

Java vs C++ 테스트 매크로 비교

기능	 JUnit 5 (Java)	 GoogleTest (C++)
테스트 정의	@Test void testName()	TEST(Suite, Name) { }
값 동등	assertEquals(expected, actual)	EXPECT_EQ(expected, actual)
조건 참	assertTrue(condition)	EXPECT_TRUE(condition)
예외 발생	assertThrows(Type.class, () -> ...)	EXPECT_THROW(expr, ExcType)
사전 실행	@BeforeEach void setUp()	void SetUp() override { }
사후 실행	@AfterEach void tearDown()	void TearDown() override { }
매개변수 반복	@ParameterizedTest @ValueSource	INstantiate_TEST_SUITE_P
모킹	Mockito.mock(MyClass.class)	Google Mock — MOCK_METHOD
커버리지	JaCoCo (mvn verify)	lcov / gcov / llvm-cov

6

정적 분석 도구 실전

Checkstyle · SpotBugs · PMD / clang-tidy · cppcheck

🕒 02:30 ~ 02:50 (20분)

정적 분석으로 코드스멜을 자동 탐지

정적 분석은 실행 없이 소스코드를 분석해 버그·스타일 위반·복잡도 문제를 찾아냅니다.

Java 정적 분석 도구

도구	탐지 내용	연동 방법
Language Support	실시간 컴파일 오류·경고	Extension Pack 내장
Checkstyle	코딩 규칙 (네이밍·포맷)	VSCode 확장 + pom.xml
SpotBugs	버그 패턴 (null·리소스·동시성)	mvn spotbugs:check
PMD	코드 품질 (중복·복잡도)	mvn pmd:check

C++ 정적 분석 도구

도구	탐지 내용	연동 방법
C/C++ Extension	실시간 IntelliSense 경고	Extension Pack 내장
clang-tidy	규칙 기반 현대화·버그 탐지	settings.json / .clang-tidy
cppcheck	런타임 오류 패턴 (null·누수)	VSCode 확장 + CMake
AddressSanitizer	메모리 오류 런타임 탐지	g++ -fsanitize=address

C++ (기존 강의)	Java (이번 강의)	공통 역할
C/C++ Extension	Language Support	실시간 경고 (에디터 저장 시)
clang-tidy	Checkstyle + PMD	코딩 규칙 검사 (커밋 전)
cppcheck	SpotBugs	버그 패턴 분석 (빌드 자동화)

정적 분석 — 클린코드 원칙 자동 점검

코드스멜 → 정적 분석 도구 → 클린코드 원칙 매핑

코드스멜	Java 도구 & 규칙	C++ 도구 & 규칙	클린코드 원칙
매직 넘버	Checkstyle: MagicNumber	clang-tidy: readability-magic-numbers	의도 표현 · 명명
빈 catch 블록	Checkstyle: EmptyBlock	clang-tidy: bugprone-empty-catch	오류 처리
미초기화 변수	SpotBugs: UwF (미초기화 필드)	cppcheck: uninitvar	안전성
null 역참조 위험	SpotBugs: NP (Null Pointer)	cppcheck: nullPointerDereference	오류 처리
미사용 코드	PMD: UnusedPrivateMethod	cppcheck: unusedFunction	죽은 코드 제거
중복 코드	PMD: CPD (복사-붙여넣기 탐지기)	cppcheck: / clang-tidy	DRY 원칙
과도한 복잡도	Checkstyle: CyclomaticComplexity	clang-tidy: readability-function-cognitive-complexity	함수 단순화
equals/hashCode 불일치	Checkstyle: EqualsHashCode	—	계약 준수
override 누락	PMD: MissingOverride / Language Support	clang-tidy: modernize-use-override	코드 명확성

💡 mvn verify (Java) / cmake --build build + cppcheck (C++) 한 번으로 모든 정적 분석을 파이프라인에서 자동 실행

7

마무리 — 실천 로드맵

단계별 적용 전략 · 핵심 요약

 02:50 ~ 02:55 (5분)

Clean Code 실천 로드맵 — 단계별 적용 전략



Boy Scout Rule: 코드를 볼 때마다 조금씩 더 깨끗하게 — Clean Code That Works!!!

전체 핵심 요약

파트	핵심 원칙	☕ Java 도구	⚙️ C++ 도구
클린코드 개념	이해하기 쉬운 코드 · 테스트로 검증	—	—
네이밍 & 스타일	의도 표현 · 일관성 · 검색 가능	Checkstyle (Google Style)	clang-format (.clang-format)
함수 & 클래스	SRP · 20줄 이내 · 추상화 통일	Language Support	C/C++ Extension
오류 처리	예외 우선 · null 반환 금지 · 의미 담기	SpotBugs (NP 탐지)	cppcheck (null 탐지)
리팩토링	코드스멜 식별 → Extract Method → 테스트	PMD (복잡도·중복)	clang-tidy (modernize-*)
단위 테스트	F.I.R.S.T · TDD 사이클 · 커버리지	JUnit 5 + JaCoCo	GoogleTest + lcov
정적 분석	빌드 파이프라인 통합 · 경고 0개 목표	mvn verify 통합	cmake + cppcheck

참고자료

Clean Code (Robert C. Martin) | Clean Code Cheat Sheet (planetgeek.ch) | Refactoring (Martin Fowler) | tango1202.github.io/categories/dev-setting/

수고하셨습니다! 🎉

Clean Code That Works!!!

📖 클린코드 = 이해하기 쉽고, 테스트 가능하고, 변경하기 쉬운 코드

📖 네이밍 — 의도를 드러내는 이름 | 스타일 — Checkstyle / clang-format 자동화

⚙️ 함수 & 클래스 — SRP · 20줄 이내 · 단일 추상화 수준

🔄 리팩토링 — 코드스멜 → Extract Method → 테스트로 안전하게

✅ TDD — ● RED → ● GREEN → ● REFACTOR | JUnit 5 / GoogleTest

🔍 정적 분석 — SpotBugs · PMD · Checkstyle / clang-tidy · cppcheck